

Notebook Shelf PRD

Modular PRD: Notebook Shelf

Table of contents

- Section 0: Claude Code upfront instructions
 - Section 1: Product overview
 - Section 2: MVP scope
 - Section 3: Technical stack
 - Section 4: Security, privacy, and data-handling requirements
 - Section 5: Git, repo, environment variables, and .gitignore
 - Section 6: Data model
 - Section 7: Module 1: Project setup and app shell
 - Section 8: Module 2: Auth
 - Section 9: Module 3: Notebook shelf
 - Section 10: Module 4: Notebook creation and customization
 - Section 11: Module 5: Notebook page editor
 - Section 12: Module 6: Links and YouTube previews
 - Section 13: Module 7: Sticky tabs
 - Section 14: Module 8: Search
 - Section 15: Module 9: Basic settings and polish
 - Section 16: Module 10: QA, testing, and security checklist
 - Section 17: Later features, not MVP
-

Section 0: Claude Code upfront instructions

Copy this into Claude Code first before giving it any module-specific instructions.

You are helping me build a new web app called Notebook Shelf.

Before writing code, read these instructions carefully and follow them throughout the project.

Project intent:

Build a personal digital notebook web app. The app should feel like a physical notebook shelf and notebook, but support typed notes, links, YouTube timestamp references, search, and sticky tabs. This is laptop-first and personal-use-first, but the architecture should leave room for multiple users later.

Core stack:

- Next.js App Router
- React
- TypeScript
- Tailwind CSS
- Supabase Auth
- Supabase Postgres
- Supabase full-text search
- New repo
- Vercel-ready later, but do not set up deployment unless requested

Build style:

- Keep the implementation lean.
- Do not add extra features beyond the current module.
- Do not over-abstract.
- Prefer simple, readable components.
- Do not install large packages unless necessary.
- Ask before adding any major dependency.
- Avoid heavy animation libraries, 3D libraries, AI features, analytics, collaboration, billing, social login, and browser extensions.
- Build one module at a time.
- After each module, stop and summarize what changed.

Security rules:

- Never hardcode secrets, API keys, database passwords, Supabase service-role keys, or private credentials.
- Use environment variables for all secrets and external config.
- Use ``.env.local`` for local secrets.
- Include ``.env.example`` with placeholder values only.
- Never expose the Supabase service-role key in frontend code.
- Use only the Supabase anon key on the client, protected by Row Level Security.
- Enable Row Level Security on all user-owned tables.
- Every user-owned table must include ``user_id``.
- Users must only be able to read, create, update, or delete their own data.
- Treat all note text, pasted links, notebook names, and metadata as untrusted input.
- Do not render arbitrary HTML from pasted links or third-party pages.
- Store safe link metadata only.
- External links should open in a new tab using safe attributes.
- Validate URLs before saving previews.
- Do not store YouTube videos, webpage HTML, screenshots, or large scraped content in MVP.
- Avoid logging personal notes or secrets to console.

Repo and file hygiene:

- Create a clean repo structure.
- Keep source files, schema files, config files, and lockfiles committed.
- Do not commit `.env`, `.env.local`, `.env.*.local`, build output, logs, dependency folders, local database files, or personal note exports.
- Create or update `.gitignore` accordingly.
- Create a README with setup instructions when project setup is complete.

Coding expectations:

- Use TypeScript.
- Keep components modular, but do not create unnecessary folders.
- Use clear names.
- Add basic loading, empty, and error states.
- Keep UI accessible with buttons, labels, keyboard navigation where relevant, and semantic HTML.
- Favor CSS/Tailwind for animations.
- Do not introduce complex state management unless needed.
- Use Supabase client helpers cleanly.
- Keep database access in a small set of helper functions where practical.

Workflow:

- When I paste a module, implement only that module.
 - Do not jump ahead to later modules.
 - If a requirement is ambiguous, make the simplest reasonable choice and note it after coding.
 - If a choice affects security or data modeling, ask before coding.
 - After completing a module, provide:
 - Files created or changed
 - What works now
 - Anything I need to configure
 - Suggested next module
-

Section 1: Product overview

Product name

Notebook Shelf

This is a working name. The app should be built so the displayed name can be changed later.

Product concept

Notebook Shelf is a personal digital notebook app that makes digital notes feel more like writing in a physical notebook.

The user opens the app and sees a forward-facing shelf of notebooks. Each notebook has a clean, aesthetic cover with no visible title on the book itself. The notebook topic appears on hover or in nearby shelf UI. Opening a notebook shows lined pages where the user types notes in a warm, curated font.

The app is designed for saving and revisiting personal notes from digital content, especially links and YouTube videos with timestamps.

Core user problem

The user does not like writing personal notes in Google Docs or Word because those tools feel sterile, awkwardly aligned, and not worth revisiting. Physical notebooks feel better, but they are bad for saving links, YouTube references, and digital sources.

Notebook Shelf should combine:

- The aesthetic pull of a physical notebook
- The usefulness of digital notes
- Searchability
- Hyperlinks
- YouTube timestamp references
- Sticky tabs for important notes

Product principle

This is not Notion. This is not Google Docs. This is not a productivity dashboard.

It is a calm, aesthetic, personal notebook system for notes the user wants to revisit.

Section 2: MVP scope

MVP features

Build these first:

- Email/password auth
- Forward-facing notebook shelf
- Create notebooks
- Edit notebook topic and style
- Delete notebook with confirmation
- Curated notebook cover colors

- Curated page colors
- 1 to 3 curated fonts
- College-ruled lined paper
- Notebook page editor for typed notes
- Auto-create next page when content passes the bottom
- Multi-page notes
- Page navigation by:
 - Clicking page edge
 - Keyboard arrow keys
 - Simple scroll, if implementation allows without complexity
- Hyperlinks
- YouTube preview cards with thumbnail shown by default
- YouTube timestamp support
- Sticky tabs attached to specific notes
- Search across notebooks
- Supabase persistence
- Supabase Row Level Security
- Basic responsive layout for laptop-first use

Explicit non-goals for MVP

Do not build these in MVP:

- Handwriting or stylus input
- Browser extension
- AI summaries
- Collaboration
- Sharing notebooks
- Team workspaces
- Billing
- Public profiles
- Mobile-first UI
- Heavy 3D animation
- Physics-based page turns
- Complex theme builder
- User-uploaded fonts
- PDF export
- Rich article scraping
- Webpage reader mode
- File uploads
- Images inside notes
- Tables

- Markdown editor unless simple formatting naturally supports it
 - Offline mode
 - Social login
 - Magic link login
-

Section 3: Technical stack

Required stack

- Framework: Next.js App Router
- Language: TypeScript
- UI: React
- Styling: Tailwind CSS
- Auth: Supabase Auth with email/password
- Database: Supabase Postgres
- Search: Supabase Postgres full-text search
- Deployment target: Vercel later

Architecture posture

The app should be:

- Single-user-first
- Multi-user-ready
- Secure by default
- Simple enough to build and debug with Claude Code
- Modular enough to expand later

App routes

Suggested route structure:

- /
 - Redirect based on auth state
- /login
 - Email/password login
- /signup
 - Email/password account creation
- /app
 - Authenticated notebook shelf
- /app/notebooks/[notebookId]
 - Notebook editor
- /app/search

- Optional dedicated search page if needed
- /app/settings
 - Optional simple settings page later

Keep routing simple. Do not create deeply nested routes unless needed.

Section 4: Security, privacy, and data-handling requirements

Security goals

The app will store personal notes. Treat user data as private and sensitive.

Required practices

- Use Supabase Auth.
- Use email/password auth only in MVP.
- Enable Row Level Security on all user-owned tables.
- Every table containing user-created data must include `user_id`.
- RLS policies must enforce that authenticated users can only access rows where `user_id = auth.uid()`.
- Do not use the Supabase service-role key in frontend code.
- Do not hardcode secrets.
- Do not log private note content.
- Do not render untrusted HTML.
- Sanitize or safely render note content.
- Validate pasted URLs.
- External links should open with:
 - `target="_blank"`
 - `rel="noopener noreferrer"`

Link preview security

For MVP:

- Store only safe metadata.
- Do not store raw webpage HTML.
- Do not inject HTML from external pages.
- Do not execute scripts from external content.
- YouTube previews should use safe metadata:
 - URL
 - Video ID
 - Title if available

- Thumbnail URL
- Timestamp
- Source
- User note

Data minimization

Store only what the app needs.

Store:

- Notebook topic
- Notebook visual style
- Page style
- Note content
- Link metadata
- YouTube metadata
- Sticky tab metadata
- Searchable text

Do not store:

- YouTube videos
- Full webpage HTML
- Large scraped article bodies
- Screenshots
- Private credentials
- Debug logs containing notes

Multi-user-ready requirement

Even though this app starts personal, do not design the database as if there will only ever be one user.

Each user-owned table should include:

- id
 - user_id
 - created_at
 - updated_at
-

Section 5: Git, repo, environment variables, and .gitignore

Repo instructions

Claude Code should initialize this as a fresh repo.

Expected setup:

- Create a new project folder.
- Initialize Git.
- Create a clean Next.js project.
- Use TypeScript.
- Use Tailwind.
- Add Supabase dependencies only when needed.
- Add a README with setup steps.

Environment variables

Use .env.local for real local values.

Expected .env.local keys:

```
NEXT_PUBLIC_SUPABASE_URL=  
NEXT_PUBLIC_SUPABASE_ANON_KEY=
```

Do not put service-role keys in the frontend.

Create .env.example:

```
NEXT_PUBLIC_SUPABASE_URL=your_supabase_project_url  
NEXT_PUBLIC_SUPABASE_ANON_KEY=your_supabase_anon_key
```

.gitignore requirements

The .gitignore should include:

```
# dependencies  
node_modules/
```

```
# next  
.next/  
out/
```

```
# production  
build/  
dist/
```

```
# env
```

```
.env
.env.local
.env.*.local

# vercel
.vercel/

# logs
*.log
npm-debug.log*
yarn-debug.log*
yarn-error.log*
pnpm-debug.log*

# os
.DS_Store
Thumbs.db

# testing and coverage
coverage/

# local database or generated data
*.sqlite
*.sqlite3
*.db

# local personal exports or backups
exports/
backups/
personal-notes/
```

Do not ignore

These should be committed:

- .env.example
- package.json
- package lockfile
- TypeScript config
- Tailwind config
- Next config
- Supabase schema or migration files
- README
- Source files

Section 6: Data model

Use this as the intended data model. Claude Code can create SQL migrations or provide SQL to run in Supabase.

Table: notebooks

Purpose: stores each notebook on the shelf.

Fields:

- id: uuid, primary key
- user_id: uuid, references auth user
- topic: text, required
- cover_color: text, required
- page_color: text, required
- line_style: text, default college_ruled
- font_style: text, required
- sort_order: integer, optional
- created_at: timestamp
- updated_at: timestamp

Notes:

- The topic should not appear directly on the notebook cover.
- The topic can appear on hover or in shelf UI.

Table: pages

Purpose: stores notebook pages.

Fields:

- id: uuid, primary key
- user_id: uuid, references auth user
- notebook_id: uuid, references notebooks
- page_number: integer, required
- created_at: timestamp
- updated_at: timestamp

Notes:

- Pages are visual containers.
- Notes can span pages.

- Page auto-creation should happen when content flows past the current page.

Table: notes

Purpose: stores typed note blocks.

Fields:

- id: uuid, primary key
- user_id: uuid, references auth user
- notebook_id: uuid, references notebooks
- start_page_id: uuid, references pages
- content: text, required
- content_plaintext: text, optional but useful for search
- position_index: integer, required
- spans_pages: boolean, default false
- created_at: timestamp
- updated_at: timestamp

Notes:

- Notes are attached to notebooks and start on a page.
- A note can visually span multiple pages.
- For MVP, position can be simple and sequential.
- Do not overbuild complex drag-positioning.

Table: links

Purpose: stores hyperlinks and YouTube references attached to notes.

Fields:

- id: uuid, primary key
- user_id: uuid, references auth user
- note_id: uuid, references notes
- url: text, required
- source_type: text, values like url, youtube
- title: text, optional
- thumbnail_url: text, optional
- youtube_video_id: text, optional
- youtube_timestamp_seconds: integer, optional
- created_at: timestamp
- updated_at: timestamp

Notes:

- Store metadata only.
- Do not store raw HTML.

Table: sticky_tabs

Purpose: stores tabs attached to specific notes.

Fields:

- id: uuid, primary key
- user_id: uuid, references auth user
- notebook_id: uuid, references notebooks
- note_id: uuid, references notes
- color: text, required
- label: text, optional
- created_at: timestamp
- updated_at: timestamp

Notes:

- Sticky tabs attach to specific notes, not just pages.
- Clicking a sticky tab should navigate to the associated note.

Search

Search should query:

- Notebook topic
- Note plaintext
- Link title
- Link URL
- YouTube title

MVP search can use Supabase Postgres full-text search or simple ilike search first, then upgrade to full-text if needed.

Section 7: Module 1: Project setup and app shell

Claude Code prompt for Module 1

Implement Module 1 only.

Goal:

Create the initial Next.js app shell for Notebook Shelf.

Requirements:

- Create a fresh Next.js project using App Router, TypeScript, and Tailwind CSS.
- Initialize Git if it is not already initialized.
- Add a clean folder structure.
- Add a README with setup instructions.
- Add `.env.example`.
- Add or update `.gitignore` using the requirements from the PRD.
- Create a basic app shell with:
 - Landing route
 - Placeholder authenticated app route at `/app`
 - Basic layout
 - Simple visual style that feels calm, warm, and notebook-like
- Do not implement auth yet.
- Do not implement Supabase yet unless required for setup.
- Do not add unnecessary dependencies.
- Do not build notebook features yet.

Acceptance criteria:

- App runs locally.
- TypeScript works.
- Tailwind works.
- `/` loads.
- `/app` loads a placeholder page.
- `.gitignore` protects env files, build output, logs, dependencies, and local exports.
- `.env.example` exists with Supabase placeholder variables.
- README includes local setup instructions.

Stop after this module and summarize changed files.

Module 1 requirements

Create the foundation only.

Suggested structure:

```
src/  
  app/  
    layout.tsx  
    page.tsx  
  app/  
    page.tsx  
  components/  
    ui/
```

lib/
styles/

Keep it simple.

Visual direction

Use:

- Warm off-white background
 - Soft shadows
 - Rounded panels
 - Calm typography
 - No heavy visuals yet
-

Section 8: Module 2: Auth

Claude Code prompt for Module 2

Implement Module 2 only.

Goal:

Add Supabase email/password authentication.

Requirements:

- Install Supabase client dependency if needed.
- Create Supabase client setup using environment variables.
- Add `/login` page.
- Add `/signup` page.
- Use email/password auth only.
- Do not add magic link.
- Do not add social login.
- Add basic logout.
- Protect `/app` so unauthenticated users are redirected to login.
- Keep auth UI simple and clean.
- Do not build notebook features yet.
- Do not hardcode Supabase credentials.
- Do not expose service-role keys.

Security:

- Use only `NEXT\_PUBLIC\_SUPABASE\_URL` and `NEXT\_PUBLIC\_SUPABASE\_ANON\_KEY` on the client.
- Never use service-role key.
- Add basic error states without logging sensitive information.

Acceptance criteria:

- User can sign up with email/password.
- User can log in.
- User can log out.
- Authenticated user can access `/app`.
- Unauthenticated user cannot access `/app`.
- `.env.example` documents required variables.
- No secrets are committed.
- App still runs locally.

Stop after this module and summarize changed files.

Auth behavior

Required:

- Sign up with email/password
- Log in with email/password
- Log out
- Auth-gated /app

Not required in MVP:

- Password reset
 - Magic link
 - OAuth
 - Profile settings
-

Section 9: Module 3: Notebook shelf

Claude Code prompt for Module 3

Implement Module 3 only.

Goal:

Build the authenticated notebook shelf UI and basic notebook listing.

Requirements:

- Create a forward-facing shelf/grid of notebooks on `/app`.
- Notebook covers should face forward, not show spines.
- Notebook topic should not appear on the cover.
- On hover, the notebook should rise slightly and show the topic label nearby.
- Include empty state for no notebooks.

- Include a button to create a notebook, but the full creation flow can be minimal or placeholder if Module 4 is not built yet.
- Use mock data only if database tables are not ready yet, but structure the code so real data can replace it cleanly.
- Keep animation lightweight using CSS/Tailwind.
- Do not add heavy animation libraries.
- Do not build page editor yet.

Visual requirements:

- Notebook covers should look aesthetic and tactile.
- Use simple rectangles with soft shadows and slight texture if easy.
- Avoid realistic 3D or expensive effects.
- Hover interaction:
 - Slight upward movement
 - Slight shadow increase
 - Topic label appears below or beside the notebook

Acceptance criteria:

- `/app` shows a shelf/grid.
- Notebook cards face forward.
- Topic is hidden until hover or shown outside the cover.
- Hover feels smooth and lightweight.
- Empty state works.
- No heavy dependencies added.

Stop after this module and summarize changed files.

Shelf behavior

The shelf should be simple.

Include:

- Forward-facing notebooks
- Hover lift
- Hover label
- Create notebook button
- Empty state

Avoid:

- Drag-and-drop
- Shelf physics
- 3D rotations
- Complex animations

- Notebook spines
-

Section 10: Module 4: Notebook creation and customization

Claude Code prompt for Module 4

Implement Module 4 only.

Goal:

Add notebook creation, editing, deletion, and curated customization.

Requirements:

- Create Supabase tables for notebooks if not already created.
- Add RLS policies so users can only access their own notebooks.
- Add notebook creation flow.
- User can define notebook topic.
- User can choose from curated cover colors.
- User can choose from curated page colors.
- User can choose from 1 to 3 curated fonts.
- User can choose college-ruled lined paper, but no need for other page styles in MVP.
- User can edit notebook topic and style.
- User can delete notebook with confirmation.
- Notebook topic must not display directly on the cover.
- Notebook data must persist in Supabase.
- Do not build page editor yet beyond linking notebook open route.

Suggested curated options:

Cover colors:

- Deep navy
- Forest green
- Clay
- Dusty rose
- Mustard
- Charcoal

Page colors:

- Warm ivory
- Soft cream
- Light parchment

Fonts:

- Warm serif

- Clean serif
- Typewriter-inspired

Security:

- Use `user\_id` on notebooks.
- Enable RLS.
- Do not let users access other users' notebooks.
- Validate required fields.
- Do not trust client-provided `user\_id` without server/session verification where applicable.

Acceptance criteria:

- Authenticated user can create a notebook.
- Created notebook appears on shelf.
- User can edit topic and style.
- User can delete notebook after confirmation.
- User cannot see notebooks from another user.
- RLS policies exist.
- No secrets are committed.

Stop after this module and summarize changed files.

Color scheme details

Use curated values. Do not create a full color picker.

Recommended token names:

cover_navy
cover_forest
cover_clay
cover_rose
cover_mustard
cover_charcoal

page_ivory
page_cream
page_parchment

font_serif_warm
font_serif_clean
font_typewriter

Notebook topic behavior

- Required field.

- Appears on shelf hover label.
 - Appears in notebook UI header.
 - Does not appear on notebook cover.
-

Section 11: Module 5: Notebook page editor

Claude Code prompt for Module 5

Implement Module 5 only.

Goal:

Build the notebook page editor for typed notes on college-ruled lined paper.

Requirements:

- Create Supabase tables for pages and notes if not already created.
- Add RLS policies for pages and notes.
- Add notebook detail route at `/app/notebooks/[notebookId]`.
- Display notebook pages with college-ruled lined paper.
- User can type notes into the notebook.
- Text should align naturally with the lined paper.
- Use the selected notebook font.
- Use the selected page color.
- Auto-create a new page when content passes the bottom of the current page.
- A single note can span multiple pages visually.
- User can navigate pages by:
 - Clicking left/right page edge controls
 - Keyboard arrow keys
- Add basic save behavior.
- Add loading and error states.
- Keep editor lean.
- Do not install a complex rich text editor unless absolutely necessary.
- Do not add formatting toolbar in MVP.
- Do not add handwriting.
- Do not add drag-and-drop note positioning.

Editor behavior:

- The experience should feel like typing into a physical notebook.
- The underlying data can be notes/blocks.
- Notes should save to Supabase.
- Use simple sequential positioning.

Security:

- User can only access pages and notes for their own notebooks.
- Enable RLS on pages and notes.
- Treat note content as untrusted text.
- Do not render arbitrary HTML.
- Avoid logging note content.

Acceptance criteria:

- User can open a notebook.
- User can type notes.
- Notes persist after refresh.
- Lined paper displays correctly.
- Font and page style reflect notebook settings.
- New page is created when needed.
- Arrow key navigation works.
- Page-edge click navigation works.
- User cannot access another user's notebook pages or notes.

Stop after this module and summarize changed files.

Editor design recommendations

Keep the first editor simple.

Preferred:

- Plain text or controlled textarea/contenteditable with careful rendering
- Notebook-like page container
- CSS lined background
- Sequential notes

Avoid:

- TipTap unless absolutely necessary
- Slate
- ProseMirror directly
- Full rich text editor
- Custom document engine
- Pixel-perfect pagination engine

Pagination guidance

Lean implementation is more important than perfect paper simulation.

Acceptable MVP behavior:

- User types in a note area.
- The app estimates page overflow based on content height.

- When overflow is detected, it creates or reveals the next page.
- Visual continuation can be simple.

Do not spend excessive effort on perfect pagination.

Section 12: Module 6: Links and YouTube previews

Claude Code prompt for Module 6

Implement Module 6 only.

Goal:

Add hyperlink support and YouTube preview cards with thumbnails shown by default.

Requirements:

- Create Supabase `links` table if not already created.
- Add RLS policies for links.
- Detect pasted URLs in note content.
- Save detected links as link records associated with the relevant note.
- Normal links should be clickable.
- External links should open in a new tab safely.
- YouTube links should be detected.
- YouTube preview should show by default.
- YouTube preview should include:
 - Thumbnail
 - Title if available
 - URL
 - Timestamp if present
- Support YouTube timestamp URLs.
- Store timestamp in seconds.
- Do not store YouTube video content.
- Do not store raw webpage HTML.
- Keep preview metadata lightweight.
- If fetching title is too complex for MVP, use the URL/video ID first and leave title as optional.

YouTube URL handling:

- Support youtube.com/watch?v=VIDEO_ID
- Supportyoutu.be/VIDEO_ID
- Support timestamp formats like `t=90`, `t=1m30s`, or `start=90` if practical.
- Thumbnail can be based on YouTube video ID using a standard thumbnail URL pattern.

Security:

- Validate URLs before saving.
- Do not render arbitrary HTML.
- Use safe external link attributes.
- Treat titles and URLs as untrusted.
- RLS must restrict link access to the owning user.

Acceptance criteria:

- Pasting a normal URL creates a clickable hyperlink.
- Pasting a YouTube URL creates a preview card.
- YouTube preview shows thumbnail by default.
- Timestamp is parsed and stored when present.
- Clicking the preview opens the YouTube link.
- Links persist after refresh.
- No raw external HTML is rendered.

Stop after this module and summarize changed files.

Recommended lean YouTube preview approach

Use the YouTube video ID to generate a thumbnail URL.

Example:

`https://img.youtube.com/vi/{VIDEO_ID}/hqdefault.jpg`

For MVP, this avoids needing the YouTube API.

Title can be:

- Manually editable later
- Fetched later through a safe metadata endpoint
- Optional in MVP

Link preview design

Show a compact card:

- Thumbnail on left
 - Title or URL on right
 - Timestamp chip if present
 - Small external-link icon or text
-

Section 13: Module 7: Sticky tabs

Claude Code prompt for Module 7

Implement Module 7 only.

Goal:

Add sticky tabs attached to specific notes.

Requirements:

- Create Supabase `sticky\_tabs` table if not already created.
- Add RLS policies for sticky tabs.
- User can add a sticky tab to a specific note.
- User can choose tab color from a curated palette.
- User can optionally add a short label.
- Sticky tabs should appear visually on the notebook page edge or side rail.
- Clicking a sticky tab jumps to the associated note.
- Add a simple tabbed/starred review view inside the notebook.
- User can remove a sticky tab.
- Do not build complex tagging system.
- Do not build drag-and-drop tab ordering.

Suggested tab colors:

- Blue
- Red
- Yellow
- Green
- Purple
- Orange

Suggested meanings, not enforced:

- Blue: concepts
- Red: important
- Yellow: revisit
- Green: examples
- Purple: ideas
- Orange: action items

Security:

- Sticky tabs must include `user\_id`.
- RLS must restrict access to the owning user.
- Labels are untrusted text and should be rendered safely.

Acceptance criteria:

- User can tab a note.
- User can choose a tab color.
- Tab appears visually.
- Clicking the tab navigates to the note.
- User can view all tabbed notes in a notebook.
- User can remove a tab.
- Tabs persist after refresh.
- Users cannot access tabs from other users.

Stop after this module and summarize changed files.

Sticky tab behavior

Tabs attach to notes, not pages.

Visual behavior:

- Small colored tabs on side edge
- Hover shows label or note preview
- Click jumps to note

Keep it simple.

Section 14: Module 8: Search

Claude Code prompt for Module 8

Implement Module 8 only.

Goal:

Add search across the user's notebooks, notes, and links.

Requirements:

- Add a search input accessible from `/app`.
- Search should cover:
 - Notebook topic
 - Note content
 - Link URL
 - Link title
 - YouTube title if present
- Results should show:
 - Matching notebook
 - Snippet of note or link
 - Source type if relevant

- Click target that opens the notebook and jumps to the relevant note where possible
- Start with Supabase Postgres full-text search if practical.
- If full-text setup becomes too heavy, use simple `ilike` search first and note the upgrade path.
- Search must only return the authenticated user's own data.
- Keep UI simple.
- Do not add Algolia, Meilisearch, or external search services in MVP.

Security:

- Search queries are untrusted input.
- Do not interpolate raw search strings into unsafe SQL.
- Use parameterized queries or Supabase query helpers.
- RLS must apply to search results.
- Do not return data from other users.

Acceptance criteria:

- User can search from app.
- Search returns notebooks, notes, and links owned by the user.
- Search results are clickable.
- Search does not expose other users' data.
- Empty and error states work.
- No external search service is added.

Stop after this module and summarize changed files.

Search recommendation

For MVP, acceptable options:

- Supabase full-text search
- Simple ilike search over note content and notebook topic

Preferred progression:

- Start with simple search if faster.
- Move to Postgres full-text search once core app works.

Do not add Algolia or Meilisearch unless search becomes a major bottleneck.

Section 15: Module 9: Basic settings and polish

Claude Code prompt for Module 9

Implement Module 9 only.

Goal:

Add basic visual polish, settings, and UX refinements without expanding scope.

Requirements:

- Add small UI refinements to make the app feel cohesive.
- Add loading states where missing.
- Add empty states where missing.
- Add confirmation dialogs for destructive actions.
- Add simple account/logout area.
- Add keyboard navigation hints where helpful.
- Improve notebook page spacing and lined-paper feel.
- Keep animations lightweight.
- Do not add new major features.
- Do not add browser extension.
- Do not add AI.
- Do not add mobile-first redesign.
- Do not add PDF export.

Acceptance criteria:

- App feels cohesive.
- Destructive actions have confirmation.
- Loading and empty states are handled.
- Keyboard navigation is discoverable.
- Page editor remains fast.
- No heavy dependencies added.

Stop after this module and summarize changed files.

Polish priorities

Prioritize:

- Speed
- Readability
- Lined-paper feel
- Clean notebook shelf
- Clear navigation
- Calm colors
- Good hover states

Avoid:

- Flashy animations
 - Too many colors
 - Too many settings
 - Complex menus
-

Section 16: Module 10: QA, testing, and security checklist

Claude Code prompt for Module 10

Implement Module 10 only.

Goal:

Review, test, and harden the MVP.

Requirements:

- Run lint/typecheck/build commands.
- Fix TypeScript errors.
- Fix obvious accessibility issues.
- Check auth-gated routes.
- Check RLS assumptions.
- Check that no secrets are committed.
- Check `.gitignore`.
- Check that `.env.example` is safe.
- Check that external links use safe attributes.
- Check that note content is not rendered as arbitrary HTML.
- Check that users can only access their own data.
- Add a short QA checklist to README.

Do not add new product features during this module.

Acceptance criteria:

- App builds successfully.
- App passes lint/typecheck if configured.
- README includes setup and QA steps.
- `.gitignore` is correct.
- No secrets are committed.
- Basic security requirements are satisfied.

Stop after this module and summarize:

- Tests/checks run
- Issues found

- Issues fixed
- Any remaining risks

Manual QA checklist

Use this list before calling MVP complete:

- Can sign up.
 - Can log in.
 - Can log out.
 - Cannot access /app while logged out.
 - Can create notebook.
 - Can edit notebook.
 - Can delete notebook with confirmation.
 - Notebook topic does not appear on cover.
 - Hover label appears on shelf.
 - Can open notebook.
 - Can type notes.
 - Notes persist after refresh.
 - Pages look lined and notebook-like.
 - Text uses chosen font.
 - Page uses chosen paper color.
 - New page is created when needed.
 - Arrow keys navigate pages.
 - Page edge clicks navigate pages.
 - Can paste normal URL.
 - Normal URL is clickable.
 - Can paste YouTube URL.
 - YouTube preview appears with thumbnail.
 - YouTube timestamp is parsed.
 - Can add sticky tab to note.
 - Can remove sticky tab.
 - Clicking sticky tab jumps to note.
 - Search finds notes.
 - Search finds links.
 - Search finds notebook topics.
 - External links open safely.
 - No secrets in repo.
 - .env.local is ignored.
 - .env.example contains placeholders only.
-

Section 17: Later features, not MVP

These are good ideas, but should not be built until the MVP works.

V1.5 candidates

- Markdown export
- JSON backup/export
- Password reset
- Better full-text search ranking
- Link title fetching through a safe backend route
- More notebook cover styles
- More page templates:
 - dotted
 - grid
 - blank
- More tab review options
- Better keyboard shortcuts

Later candidates

- Browser extension for saving YouTube timestamps and highlighted article text
- PDF export that preserves notebook look
- Mobile read-only mode
- iPad/tablet polish
- Image attachments
- Screenshot clipping
- Article clipping
- AI-generated summaries
- AI search over notes
- Sharing
- Collaboration
- Public notebooks
- Custom uploaded fonts

Browser extension later

The browser extension could eventually let the user:

- Save current YouTube video
- Save current timestamp
- Save selected article text
- Choose notebook destination
- Add a quick note
- Send it directly into Notebook Shelf

Do not build this in MVP.

Suggested build order

Use this order with Claude Code:

- Paste Section 0 first.
- Then paste Section 7: Module 1.
- Test locally.
- Then paste Section 8: Module 2.
- Test auth.
- Then continue module by module.

Do not paste the whole PRD at once unless you want Claude Code to reason through the full plan. For implementation, paste one module at a time.